

PART I — What ArchAngel optimization REALLY is

The truth about “Fast Genetic Optimization”

When you run:

- fast genetic algorithm
- 4000 output files
- ranked by profit / FT / win rate

you are **not finding strategies**.

You are finding **parameter clusters that exploited a specific historical regime**.

That's fine.

But only if you understand **what regime that was** and **what strategy archetype those parameters represent**.

Right now, your process is implicitly assuming:

“The future will statistically resemble the backtest window.”

That assumption is false more often than it is true, especially for grid systems.

PART II — Why “it worked yesterday, not today”

This is the key failure mode:

Optimization silently locks onto ONE dominant regime

Example:

- Backtest window = 3 months
- That window was:
 - low ADX
 - compressed volatility
 - clean mean reversion

The optimizer then:

- tightens pip step
- increases lot exponent
- loosens RSI / BB filters

Result:

- beautiful equity curve
- explosive payout speed
- **zero regime robustness**

The moment:

- ADX rises
- ATR expands
- momentum persists

the setfile does exactly what it was designed to do:

die.

This is not a bug.

It is **exactly what you asked the optimizer to produce.**

PART III — The missing layer: Strategy Identity

Every setfile **is a strategy**, whether you label it or not.

Before optimization, you must answer:

“What type of strategy am I optimizing for?”

There are only **4 valid identities** for ArchAngel setfiles:

Strategy Type 1 — Range Mean Reversion (Classic Grid)

Indicators

- RSI ON (extremes)
- BB ON (touches)
- ADX filter: LOW only
- EMA: OFF or permissive

Grid

- tight pip step
- moderate lot exponent
- high trade count

Regime

- $ADX < 20$
- ATR compressed
- no macro events

What optimization finds

- high profit
 - high win rate
 - high tail risk
-

Strategy Type 2 — Defensive Mean Reversion (Survivor Grid)

Indicators

- RSI ON
- BB ON
- ADX strict filter
- EMA permissive or trend-aligned

Grid

- wider spacing
- low exponent
- capped orders

Regime

- mixed / uncertain
- regime transition periods

Optimization goal

- smooth equity

- low DD
 - boring profits
-

Strategy Type 3 — Trend-Compatible Grid (Hybrid)

Indicators

- EMA ON (trend direction)
- ADX ON (trend strength)
- RSI pullbacks only
- BB = avoid extreme

Grid

- shallow
- low max orders
- fast TP

Regime

- trending indices
- NQ bull runs
- macro momentum

Optimization goal

- survive trends
- avoid countertrend death

Strategy Type 4 — Event / Volatility-Adaptive

Indicators

- ATR ON for scaling
- filters stricter
- delays ON

Grid

- wide spacing
- very low scaling

Regime

- news weeks
- volatility expansion

Optimization goal

- capital preservation

 If you optimize without explicitly choosing ONE of these identities, the genetic algorithm will **mix them**, producing Frankenstein files that look amazing and fail fast.

PART IV — How optimization SHOULD be run (this is critical)

Step 1 — Regime-lock your backtest window

Before you optimize, classify the period:

- ADX distribution (H1)
- ATR vs long-term median
- major macro events?

Example labels:

- “Low-vol range”
- “Trend expansion”
- “Transition / chop”

You do **not** optimize across mixed regimes unless you are explicitly building survivors.

Step 2 — Constrain the optimizer HARD

This is the most important thing you are not doing yet.

Bad optimization:

- 20 parameters free
- wide ranges
- optimizer chooses everything

Good optimization:

- only **3–6 parameters mutable**
- the rest LOCKED to strategy identity

Example:

If you're optimizing a range MR strategy:

- ADX thresholds locked
- EMA logic locked
- grid exponent range narrow
- pip step bounded

This forces the optimizer to explore **within a strategy**, not across strategies.

Step 3 — Generate files, but don't rank them by profit first

Your current ranking:

- BT profit
- FT profit
- win rate
- trade count

This is backwards.

Correct ranking order for grid systems:

1. **Max drawdown / equity stability**
2. **Trade count sufficiency** (no under-sampling)
3. **Profit factor**
4. **Profit**

Profit is LAST, not first.

Why:

- grid systems fail by DD, not by low profit.
-

Step 4 — Cluster, don't cherry-pick

Out of 4000 files, you should:

- cluster by similarity (pip step, exponent, filters)
- identify **parameter neighborhoods** that work

If 1 file works → luck

If 20 similar files work → signal

Your setfile finder should be used for **cluster identification**, not top-1 selection.

PART V — Why forward testing disagrees with backtesting

This is normal and expected.

Backtest answers:

“Did this work in that regime?”

Forward test answers:

“Is that regime still here?”

If they disagree, the conclusion is NOT:

- “optimization is bad”

The conclusion is:

- **regime shifted**

- wrong strategy identity deployed
-

PART VI — Turning this into a real pipeline

Here is the correct pipeline you are aiming for:

Weekly process

1. Identify current regime per instrument
2. Activate only setfiles designed for that regime
3. Park the rest

Monthly process

1. Run optimizations per regime bucket
2. Refresh survivor + accelerator pools
3. Retire fragile files

Always

- Never rely on a single file
- Always have replacements pre-validated

This is how you achieve **3–5% monthly compounding**, not by chasing the top optimizer result.

PART VII — Why F files don't work

Your instinct is right.

F files usually fail because:

- they are optimized on micro-regimes
- they over-tighten grid parameters
- they rely on mean reversion that no longer exists

Comparing them to winners:

- look at ADX exposure
- look at pip step density
- look at max order count

You will almost always see:

Winners survived worse regimes.
Losers extracted more from easier regimes.

PART VIII — The correct mental shift

Stop asking:

“Which file is best?”

Start asking:

“Which strategy fits THIS regime?”

ArchAngelX is not a money printer.

It is a **strategy deployment framework**.

You are becoming:

- part trader
- part quant

- part ops engineer

That is why this feels complex.

It is complex. But now it is **structured**.